

Tuần 2: Trình sát Mạng & Kỹ thuật Quét Cổng (Network Reconnaissance & Port Scanning)

Mục Tiêu / Objectives

Trong tuần 2, chúng ta sẽ bước vào giai đoạn đầu tiên của mọi cuộc kiểm thử bảo mật (Penetration Testing): Trình sát (Reconnaissance). Học viên sẽ tự tay xây dựng một công cụ rà quét cổng mạng (Port Scanner) bằng Python để tìm ra các "cánh cửa" đang mở trên hệ thống, từ đó đánh giá các điểm yếu tiềm ẩn. / In week 2, we enter the first phase of any penetration test: Reconnaissance. Students will build a Port Scanner using Python to find open "doors" on a system, thereby assessing potential vulnerabilities.

Mục tiêu cụ thể / Specific Objectives:

1. Hiểu rõ nguyên lý hoạt động của kỹ thuật Port Scanning (TCP Connect).
2. Nắm vững danh sách các Port thông dụng và dịch vụ đi kèm (80, 443, 21, 22, 3306).
3. Lập trình công cụ Scanner bằng Python qua 3 cấp độ: Quét 1 cổng -> Quét dải cổng (Vòng lặp) -> Quét đa luồng tốc độ cao (Multi-threading).
4. Khắc sâu nguyên tắc Đạo đức: Tuyệt đối chỉ quét hệ thống Localhost.

Lý Thuyết / Theory (with definitions and examples)

1. Trình sát mạng (Reconnaissance) là gì?

- Giống như việc một thám tử khảo sát ngôi nhà trước khi quyết định cách đột nhập. Hacker (hoặc chuyên gia bảo mật) sẽ dò tìm xem mục tiêu đang chạy hệ điều hành gì, mở những cổng (Port) nào, và phần mềm nào đang lắng nghe sau các cổng đó.
- Mục đích: Tìm kiếm lỗ hổng (Ví dụ: Tìm thấy cổng 21 - FTP mở mà không yêu cầu mật khẩu).

2. Kỹ thuật Port Scanning

- Một máy tính có **65535** cổng.
- Công cụ Scanner sẽ lần lượt gõ cửa từng cổng.

TCP Connect Scan: Phương pháp cơ bản nhất (mà chúng ta sẽ code hôm nay). Máy của bạn cố gắng hoàn thành "Bắt tay 3 bước" (3-way handshake) với máy đích.

- Nếu cổng MỞ: Máy đích phản hồi **SYN-ACK**. Kết nối thành công.
- Nếu cổng ĐÓNG: Máy đích phản hồi **RST** (Reset). Kết nối thất bại.
- *Lưu ý:* Kỹ thuật này rất ồn ào và dễ bị Tường lửa (Firewall) hoặc hệ thống phát hiện xâm nhập (IDS) ghi lại log.

2b. Các kiểu quét cổng theo CEH & 6 cờ TCP (CEH Module 03)

CEH phân loại nhiều kiểu quét, mỗi kiểu lợi dụng cách khác nhau mà TCP phản hồi **6 cờ (flags)**: **SYN**, **ACK**, **FIN**, **RST**, **PSH**, **URG**. Chúng ta code TCP Connect Scan, nhưng bạn cần biết cả họ để đọc đề thi và hiểu vì sao hacker thật hiếm khi dùng Connect Scan:

Kiểu quét	Gói gửi đi	Nếu cổng MỞ	Ồn ào?	Ghi chú
TCP Connect (-ST)	Bắt tay 3 bước đầy đủ	Hoàn tất handshake	Rất ồn	Ta code hôm nay; luôn bị log
SYN / Stealth (-SS)	Chỉ SYN	Nhận SYN-ACK rồi gửi RST (không hoàn tất)	Ít ồn hơn	Kiểu "mặc định" của hacker (Nmap, Tuần 5)
FIN (-SF)	Chỉ FIN	Không phản hồi	Lén	Né được firewall đơn giản
XMAS (-SX)	FIN+PSH+URG	Không phản hồi	Lén	"Cây thông" vì bật nhiều cờ
NULL (-SN)	Không cờ nào	Không phản hồi	Lén	Không hiệu quả với Windows
UDP (-SU)	Gói UDP	Thường im lặng	Chậm	Cho DNS, SNMP...

Ba trạng thái cổng (Port States) theo CEH — quan trọng cho Tuần 5:

- **Open (Mở):** có dịch vụ đang lắng nghe.
- **Closed (Đóng):** không có dịch vụ, nhưng máy vẫn trả lời (**RST**).
- **Filtered (Bị lọc):** firewall nuốt gói tin, không có phản hồi → ta không biết mở hay đóng.

Scanner tự viết của ta chỉ phân biệt được Open/Closed. Muốn thấy "Filtered" và dùng SYN scan, cần quyền root và gói tin thô — đó là lý do Tuần 5 chuyển sang Nmap.

3. Đa luồng (Multi-threading) là gì?

- Nếu dùng một vòng lặp bình thường để quét 65535 cổng, và mỗi cổng mất 1 giây để chờ phản hồi (timeout), bạn sẽ mất hơn 18 tiếng!
- **Đa luồng (Threading)** cho phép Python mở hàng chục, hàng trăm "công nhân" (threads) đi gõ cửa các cổng cùng một lúc, rút ngắn thời gian quét xuống chỉ còn vài phút hoặc vài giây.

Cảnh Báo An Toàn & Đạo Đức / Safety Warnings and Ethical Notices

[!WARNING]

CẢNH BÁO PHÁP LÝ & ĐẠO ĐỨC (LEGAL & ETHICAL WARNING):

1. Hành động quét cổng (Port Scanning) vào một mục tiêu lạ **có thể bị coi là hành vi tấn công trình sát**, vi phạm nghiêm trọng chính sách bảo mật của các tổ chức và luật An ninh mạng.
2. Công cụ Nmap hay script bạn viết ra **CHỈ ĐƯỢC PHÉP** chạy với đích đến là **127.0.0.1** (Localhost) hoặc các máy ảo do chính bạn lập ra để phục vụ việc học.
3. Tuyệt đối không thử quét IP của trường học, công ty, hay bất kỳ trang web công cộng nào.

Thực Hành Code / Hands-On (Từ Cơ Bản Đến Phức Tạp)

Chúng ta sẽ trải qua 3 cấp độ để xây dựng một cỗ máy quét (Scanner) hoàn chỉnh. Ở bài thực hành này, trước khi quét, bạn nên mở sẵn vài Server (như file `basic_server.py` ở tuần 1) để có cổng mở cho Scanner tìm thấy nhé!

Cấp độ 1: Scanner Cơ bản (Quét 1 Cổng)

Mục tiêu: Dùng hàm `connect_ex()` thay vì `connect()`. Hàm này không làm văng lỗi (crash) khi cổng đóng, mà chỉ trả về mã lỗi.

`01_basic_scanner.py`

```
import socket # Khai báo m...c tiêu an toàn (Luôn là localhost) target_ip = "127.0.0.1" port_to_scan = 9999 # Tạo socket TCP scanner = socket.socket(socket.AF_INET, socket.SOCK_STREAM) scanner.settimeout(1) # Ch...ch...t...i...a 1 giây ...ph...n h...i print(f"Đang gõ cửa cổng {port_to_scan} trên {target_ip}...") # connect_ex tr...v...s...0 n...u k...t n...i thành công (C...ng M...) # Tr...v...các s...khác (ví d...: 61, 111) n...u C...ng ...ÓNG result = scanner.connect_ex((target_ip, port_to_scan)) if result == 0: print(f"[+] C...NG {port_to_scan}: M... (OPEN)") else: print(f"[-] C...NG {port_to_scan}: ...ÓNG (CLOSED)") scanner.close()
```

Cấp độ 2: Vòng lặp Scanner (Quét Dải Cổng)

Mục tiêu: Quét tự động từ cổng 1 đến 100 bằng vòng lặp `for`.

02_loop_scanner.py

```
import socket import time target_ip = "127.0.0.1" print(f"=== BẮT ĐẦU QUÉT HOST: {target_ip} ===") start_time = time.time() # Quét các cổng từ 1 đến 100 for port in range(1, 101): scanner = socket.socket(socket.AF_INET, socket.SOCK_STREAM) scanner.settimeout(0.1) # Chờ 0.1s mỗi cổng để quét nhanh hơn result = scanner.connect_ex((target_ip, port)) if result == 0: print(f"[+] PHÁT HIỆN CẢNG MỞ: {port}") scanner.close() end_time = time.time() print(f"Hoàn tất quét trong {round(end_time - start_time, 2)} giây.")
```

Cấp độ 3: Scanner Tốc độ cao (Multi-threading)

Mục tiêu: Sử dụng thư viện `threading` để quét hàng ngàn cổng cực nhanh.

03_fast_scanner.py

```
import socket import threading import time target_ip = "127.0.0.1" open_ports = [] # Danh sách lưu các cổng đang mở def scan_port(port): """Hàm quét 1 cổng duy nhất, sẽ thực hiện các công việc (lưu) ghi.""" scanner = socket.socket(socket.AF_INET, socket.SOCK_STREAM) scanner.settimeout(0.5) try: result = scanner.connect_ex((target_ip, port)) if result == 0: print(f"[+] Mở: Cổng {port}") open_ports.append(port) except Exception: pass # Bỏ qua lỗi finally: scanner.close() print(f"=== MULTI-THREAD SCANNER ĐANG CHẠY TRÊN {target_ip} ===") start_time = time.time() threads = [] # Danh sách quản lý các công việc # Quét 1000 cổng đầu tiên (1 - 1000) for port in range(1, 1001): # Tạo một luồng (thread) mới và giao nhiệm vụ chạy hàm scan_port t = threading.Thread(target=scan_port, args=(port,)) threads.append(t) t.start() # Ra lệnh cho công việc bắt đầu làm việc # Khi tất cả công việc làm việc xong thì mới kết thúc chương trình for t in threads: t.join() end_time = time.time() print("\n" + "="*40) print(f"BÁO CÁO KẾT QUẢ:") print(f"- Tổng số cổng mở: {len(open_ports)} {open_ports}") print(f"- Thời gian hoàn thành: {round(end_time - start_time, 2)} giây") print("="*40)
```

Bài Tập Thực Hành / Exercises

Bộ bài tập ngắn đi kèm tuần này nằm ở [week02_exercises.md](#), gồm 2 nhóm:

- **Nhóm A — Một máy (Localhost):** 3 bài [21...23_ex_*](#) trong [week02_code/](#)
 1. Checklist dịch vụ (~15 phút) — ôn Cấp độ 1 + 2
 2. Đấu tốc độ: vòng lặp vs đa luồng (~20 phút) — ôn Cấp độ 3
 3. Báo cáo kiểm toán mini (~20 phút) — Banner Grabbing + tư duy Blue Team
- **Nhóm B — Hai máy cùng mạng LAN:** 3 bài [31...33_lan_*](#) trong [week02_code/](#)
 1. Bắt liên lạc (~20 phút) — hiểu `bind("0.0.0.0")` vs `bind("127.0.0.1")`
 2. Điềm danh thiết bị trong nhà (~25 phút) — host discovery đa luồng
 3. Song đấu tường lửa (~30 phút) — Red Team vs Blue Team, kiểm chứng

bản vá

Hướng dẫn chạy trên 2 MacBook cùng Wi-Fi:
[huong_dan_lab_2_macbook.md](#)

[!WARNING]
Nhóm B chỉ được thực hiện giữa **hai máy của chính bạn trên mạng riêng ở nhà hoặc phòng lab**, và phải được chủ mạng đồng ý. Tuyệt đối không chạy ở Wi-Fi trường học, công ty, ký túc xá hay mạng công cộng.

Bài Về Nhà / Homework

Đề bài: Thám tử Banner (Banner Grabbing)

Banner Grabbing là kỹ thuật thu thập thông tin phần mềm đang chạy đằng sau một cổng mở. Ví dụ: Cổng 80 mở, nhưng đằng sau nó là Nginx hay Apache?

Dựa trên Cấp độ 3, hãy viết thêm một tính năng: Khi phát hiện cổng MỞ, thay vì chỉ in ra "[+] MỞ", hãy thử gửi một chuỗi văn bản bất kỳ vào cổng đó (ví dụ: "HELLO\\r\\n"), sau đó dùng `recv(1024)` để xem phần mềm đằng sau phản hồi lại nội dung gì. In phản hồi đó ra màn hình (Lưu ý dùng `try...except` vì có dịch vụ mở nhưng không phản hồi).

Yêu cầu kỹ thuật / Technical Requirements:

1. Code đa luồng chạy mượt mà.
2. Có hàm thu thập Banner `grab_banner(ip, port)`.
3. Chỉ thực hành quét `127.0.0.1`.

Cách nộp bài / How to Submit:

Nộp file `banner_scanner.py` lên hệ thống LMS kèm theo hình ảnh chụp Terminal hiển thị rõ một dịch vụ đã trả về Banner (Gợi ý: Mở sẵn `secure_server.py` của Tuần 1 làm "mồi" để quét, vì nó có cài đặt logic phản hồi lại Client).

Đánh Giá / Assessment Rubric Table

Tiêu chí / Criteria	Xuất sắc / Excellent (90-100%)	Tốt / Good (70-89%)	Cần cố gắng / Needs Improvement (<70%)
1. Tuân thủ An toàn	Chỉ quét <code>127.0.0.1</code> . (30 điểm)	Dùng localhost nhưng thiếu cẩn thận. (20 điểm)	Quét IP mạng ngoài/LAN. (0 điểm, FAIL).

2. Tốc độ & Đa luồng	Quét hàng ngàn cổng nhanh chóng, không bị treo nhờ threading . Đóng luồng gọn gàng bằng join() . (30 điểm)	Dùng vòng lặp thường hoặc Thread bị treo do cấu trúc sai. (15 điểm)	Không thể quét nhiều cổng. (0 điểm)
3. Thu thập Banner	Gửi thông điệp, bắt được phản hồi của phần mềm đích, xử lý ngoại lệ recv() mượt mà. (40 điểm)	Có hứng dữ liệu nhưng không xử lý ngoại lệ Timeout khiến luồng bị kẹt. (25 điểm)	Cổng mở nhưng không bắt được Banner. (10 điểm)

Mở Rộng: Xây Dựng Công Cụ Quản Trị Mạng & Phòng Thủ (Defensive Auditing)

Thay vì dùng Scanner để tấn công, chúng ta có thể sử dụng chính công cụ này với **tư duy phòng thủ (Blue Team)** để kiểm kê an ninh thiết bị. Mục tiêu là phát hiện các cổng đang mở ngoài ý muốn và đưa ra khuyến nghị đóng các dịch vụ không cần thiết để giảm thiểu rủi ro.

Một quy trình quản trị an toàn bao gồm:

1. Quét thiết bị (Localhost).
2. Kiểm tra các cổng phổ biến (Common Ports).
3. Hiển thị ý nghĩa của từng cổng.
4. Đánh giá mức độ rủi ro.
5. In ra hướng dẫn (Windows Firewall, ufw, router, v.v.) để đóng cổng.

Mã Nguồn Công Cụ: [12_defensive_auditor.py](#)

Công cụ này được thiết kế để chỉ quét an toàn trên **127.0.0.1** (theo quy định của khóa học), liệt kê các dịch vụ đang chạy và tự động đưa ra các lời khuyên bảo mật. Bạn có thể xem mã nguồn tại thư mục [week02_code](#).

Hướng Dẫn Đóng Cổng (Remediation)

Sau khi công cụ chỉ ra các cổng đang mở, người dùng cần tự đóng cổng trên thiết bị của họ:

Cổng	Khuyến nghị phòng thủ
21	Tắt dịch vụ FTP nếu không sử dụng

22	Chỉ cho phép SSH bằng khóa công khai
23	Nên tắt hoàn toàn (Dịch vụ lỗi thời)
80/443	Kiểm tra web server có cần thiết không
445	Tắt chia sẻ file nếu không dùng
3389	Giới hạn truy cập bằng firewall/VPN

Ví dụ đóng cổng trên Windows (Powershell):

```
New-NetFirewallRule -Displayname "Block RDP" -Direction Inbound -Protocol TCP -LocalPort 3389 -Action Block
```

Ví dụ đóng cổng trên Ubuntu (Linux):

```
sudo ufw deny 3389 sudo ufw deny 23 sudo ufw status
```

Quy Trình Chuẩn Của Chuyên Gia Phân Tích (Auditor):

```
Quét thiết bị ↓ Hiện thị công đang mở ↓ Ghi lại thích chí năng ↓ Đánh giá mức độ rủi ro ↓ Đề xuất hành động khắc phục (Remediation)
```

Cách tiếp cận này giúp học viên tự kiểm tra và tăng cường an toàn cho chính thiết bị của mình một cách chủ động và có trách nhiệm!

Case Study Thực Tế: Đánh Giá An Ninh Máy Tính Cá Nhân (PostgreSQL & macOS Firewall)

Dưới đây là một bài học thực tế về cách đánh giá bảo mật trên một máy tính Mac đang chạy cơ sở dữ liệu PostgreSQL.

Tổng hợp hiện trạng

Hạng mục	Trạng thái
PostgreSQL	Chỉ mở trên localhost
Cổng 5432	Không lộ ra Wi-Fi
Cổng 5432	Không lộ ra Internet
Unix Socket	Hoạt động bình thường

macOS Firewall	Đang tắt
----------------	----------

Log quan trọng:

```
Firewall is disabled. (State = 0)
```

Điều này có nghĩa là hiện tại macOS không bật tường lửa ứng dụng. Tuy nhiên, do PostgreSQL chỉ lắng nghe trên `127.0.0.1:5432` và `:::1:5432` nên riêng PostgreSQL vẫn an toàn.

Đánh giá rủi ro

Kịch bản cần lưu ý:

Nếu sau này bạn cài thêm các dịch vụ mở cổng công khai (Bind ra `0.0.0.0`) như Node.js server (`0.0.0.0:3000`), Docker containers, Redis (`6379`), MongoDB (`27017`), SSH (`22`)... thì khi Firewall tắt, các dịch vụ đó có thể bị truy cập trái phép từ mạng nội bộ Wi-Fi (ví dụ `192.168.1.15:3000`).

Khuyến nghị & Cách kiểm tra bằng lệnh (Command-line Auditing)

Đối với máy lập trình viên (developer machine), cách tốt nhất là duy trì thói quen kiểm tra định kỳ:

1. Bật macOS Firewall:

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --setglobalstate on
```

2. Xác nhận trạng thái Firewall:

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate #  
Kết quả mong muốn: Firewall is enabled. (State = 1)
```

3. Liệt kê tất cả dịch vụ đang lắng nghe trên máy (Lệnh Cục Kỳ Quan Trọng):

```
lsof -i -P | grep LISTEN
```

Lệnh này mạnh tương đương với việc chạy công cụ Port Scanner trên chính máy bạn.

4. Tìm các dịch vụ đang lộ ra toàn mạng Wi-Fi/Internet:

```
lsof -i -P | grep LISTEN | grep -v "127.0.0.1"
```

Nếu chỉ thấy `localhost`, `127.0.0.1` hoặc `:::1` thì bạn đang ở trạng thái an toàn tuyệt đối.

Đánh giá điểm bảo mật (Security Score)

Nếu hệ thống của bạn vượt qua bài kiểm tra cuối cùng:
Tiêu chí	Kết quả
Chỉ có 1 cổng mở	Có
Cổng mở là localhost	Có
Firewall bật	Có
Không có Telnet/FTP/RDP	Có
Không lộ dịch vụ ra Wi-Fi	Có
Không lộ dịch vụ ra Internet	Có

Overall Security Score: 10/10 | Risk Level: VERY LOW

Đối với một máy phát triển cá nhân, đây là một cấu hình nguyên tắc "default deny" (mặc định từ chối) hoàn chỉnh. Hãy duy trì thói quen dùng `lsof` sau khi cài đặt phần mềm mới để luôn làm chủ sự an toàn của thiết bị!

Phụ Lục: Phân Tích Chuyên Sâu Các Tiến Trình Hệ Thống (macOS Services)

Khi bạn chạy lệnh `lsof -i -P | grep LISTEN`, ngoài PostgreSQL ra, bạn có thể sẽ bắt gặp một số cổng khác đang mở. Dưới đây là phân tích chi tiết về các dịch vụ thường gặp trên macOS để giúp bạn nhận diện đâu là rủi ro, đâu là bình thường:

Phân loại kết quả mẫu

Port	Process	Phạm vi	Đánh giá
5000	ControlCenter	*	macOS AirPlay
7000	ControlCenter	*	macOS AirPlay
49152	rapportd	*	Dịch vụ Apple
54999	rapportd	*	Dịch vụ Apple
55000	rapportd	*	Dịch vụ Apple
5432	postgres	localhost	An toàn
8080	node	localhost	An toàn
49196-49992	language_server	localhost	An toàn
61034	VS Code	localhost	An toàn

1. **ControlCenter** (Port 5000, 7000)

Đây là tiến trình của macOS liên quan đến AirPlay, Screen Mirroring.

Nếu bạn thấy **TCP *:5000 (LISTEN)**, dấu * nghĩa là nó đang mở ra toàn mạng Wi-Fi.

- **Xử lý:** Nếu bạn không dùng AirPlay, hãy vào **System Settings > General > AirDrop & Handoff** và tắt **AirPlay Receiver**.

2. **rapportd** (Các Port ngẫu nhiên > 49000)

Đây là dịch vụ chính thức của Apple dùng cho Handoff, Universal Clipboard (Copy máy này Paste máy kia). Nó kết nối liên tục giữa MacBook, iPhone và iPad.

- **Xử lý:** Đây không phải mã độc. Nếu bạn dùng hệ sinh thái Apple, hãy giữ nguyên.

3. Các tiến trình lập trình nội bộ (**node**, **language_server**, **Code H**)

Khi dùng VS Code (hoặc Cursor), nó sẽ tự động bật các Language Server (như Python, TypeScript) để phân tích code, mở các cổng ngẫu nhiên (VD: 61034) trên **localhost**.

- **Xử lý:** Hoàn toàn bình thường và an toàn vì chúng bị trói chặt vào **localhost**.

Sơ Đồ An Ninh Lý Tưởng

Internet	X	(Bị chặn bởi Tường lửa mạng)		macOS Firewall	
-----			localhost	Wi-Fi	
AirPlay, Apple				5432, 8080, VSCode	

Kết luận: Không có dấu hiệu của mã độc hoặc dịch vụ bất thường. Những cổng mở ra mạng nội bộ đều thuộc dịch vụ hệ thống của Apple, còn toàn bộ dịch vụ lập trình (Database, Web server, Editor) đều được giới hạn ở **localhost**. Đây là một bức tranh bảo mật chuẩn mực!

□ Góc Nhìn CEH / CEH Alignment

Mục này gắn kiến thức Tuần 2 vào khung chuẩn CEH. Xem bản đồ tổng ở [CEH_alignment.md](#).

Ảnh xạ CEH (CEH Mapping)

Hạng mục	Nội dung
Module CEH	M03 Scanning Networks (trọng tâm) · mở đầu M04 Enumeration (Banner Grabbing)
Giai đoạn tấn công	Scanning (giai đoạn 2/5) — và Banner Grabbing bắc cầu sang giai đoạn nhận diện dịch vụ

Vai trò	Red (port scanner) và Blue (<code>12_defensive_auditor.py</code> , kiểm kê bằng <code>lsof</code>)
---------	--

Methodology — Quy trình quét chuẩn CEH

CEH dạy scanning theo trình tự có hệ thống, và các bài Tuần 2 đi đúng thứ tự này:

1. Host Discovery → máy còn sống không? (<code>lan_ex01: is_alive / "TCP ping"</code>)
2. Port Scanning → cổng nào mở? (<code>basic → loop → fast scanner</code>)
3. Service/Version → dịch vụ gì đang sau cổng? (Banner Grabbing, bài về nhà)
4. OS Fingerprinting → hệ điều hành gì? (giới thiệu, làm sâu Tuần 5 với Nmap)
5. Vulnerability Map → lỗ hổng nào? (<code>07_vulnerability_lookup</code> – tra CVE)

Từ **Scanning** sang **Enumeration**: quét cho biết "cổng 22 mở", còn Banner Grabbing (Enumeration) cho biết "đó là OpenSSH 8.9" — chi tiết này mới tra được CVE để tấn công.

Thuật ngữ CEH cần thuộc (Key Terminology)

Tiếng Việt	English	Trong Tuần 2
Trình sát chủ động	Active Reconnaissance	Quét cổng có chạm mục tiêu
Quét lén	Stealth / SYN Scan	Kiểu quét không hoàn tất handshake
Trạng thái cổng	Port State	Open / Closed / Filtered
Thu thập biểu ngữ	Banner Grabbing	Mọi tên & phiên bản phần mềm
Liệt kê	Enumeration	Bước sau Scanning, mọi chi tiết dịch vụ
Dấu vân tay HĐH	OS Fingerprinting	Đoán hệ điều hành qua đặc điểm gói tin
Bề mặt tấn công	Attack Surface	Mỗi cổng mở làm bề mặt rộng thêm

Câu hỏi ôn thi kiểu CEH (Exam-Style Questions)

1. Kiểu quét nào hoàn tất trọn vẹn bắt tay 3 bước, nên đáng tin cậy nhất nhưng cũng dễ bị IDS ghi log nhất?

- A. SYN Scan B. **TCP Connect Scan** C. FIN Scan D. NULL Scan

2. Một cổng trả về trạng thái **Filtered** nghĩa là gì?

Đáp án: Firewall đã chặn/nuốt gói tin dò, nên scanner không nhận được phản hồi và không xác định được cổng mở hay đóng.

3. Trong họ 6 cờ TCP, XMAS Scan bật những cờ nào?

*Đáp án: **FIN + PSH + URG** (nên gọi là "cây thông Noel" vì "sáng đèn" nhiều cờ).*

4. Banner Grabbing thuộc giai đoạn nào trong 5 giai đoạn tấn công, và vì sao nó quan trọng hơn việc chỉ biết cổng mở?

Đáp án: Thuộc bước Enumeration (nối tiếp Scanning). Biết phiên bản phần mềm mới tra được CVE cụ thể để chọn exploit — chỉ biết "cổng mở" thì chưa tấn công được.

5. Vì sao hacker thật thường dùng SYN Scan (**-ss**) thay cho Connect Scan (**-sT**)?

*Đáp án: SYN Scan không hoàn tất handshake (gửi **RST** sau khi nhận **SYN-ACK**), nên nhiều hệ thống không ghi lại thành một kết nối hoàn chỉnh → ít để lại dấu vết hơn.*